Server System Architectures

1 Overview

Server systems are internally organized to manage computation and data access efficiently. They are broadly categorized into **Transaction Servers** and **Data Servers**.

Architectural Comparison

Feature	Transaction Server	Data Server
Interface	SQL, specialized APIs (JDBC/ODBC)	Files, Pages, or Objects
Logic Location	Executed on Server	Executed on Client
Communication	Results shipped to client	Raw data units shipped to client
Usage	Most relational databases	Object-oriented, high-volume CAD

2 Transaction-Server (Query-Server) Architecture

The **transaction-server architecture** is the most widely used model today. It utilizes a multi-process structure accessing shared memory for high performance.

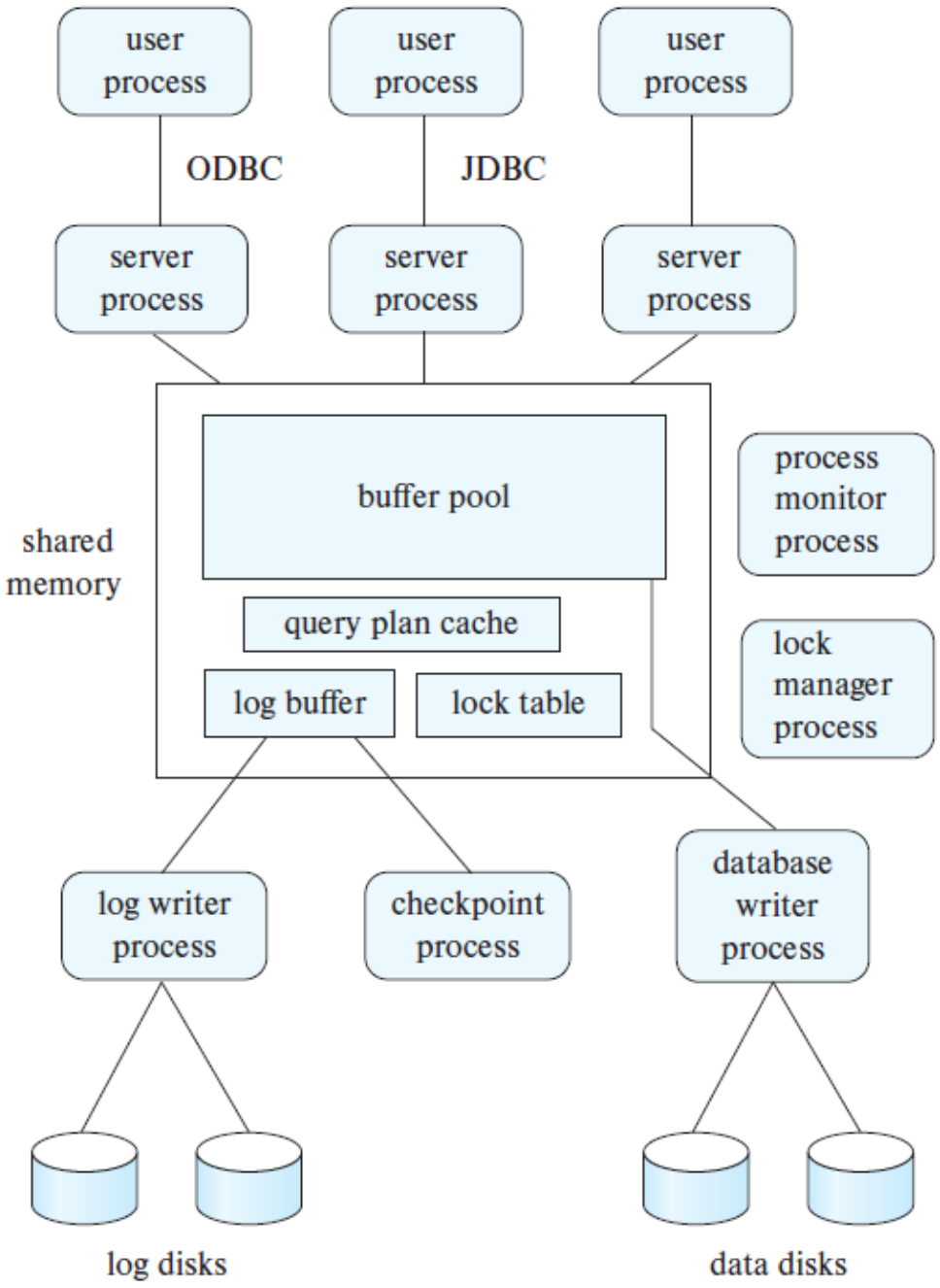

Figure 1: Shared memory and process structure.

2.1 Database System Processes

- **Server Processes:** Handle incoming queries, execute transactions, and return results.
- **Lock Manager:** Manages lock grants, releases, and deadlock detection.
- **Database Writer:** Continuously flushes modified (dirty) buffer blocks to disk.
- **Log Writer:** Moves log records from the memory buffer to stable storage.
- **Checkpoint Process:** Periodically synchronizes memory state with disk state.

- **Process Monitor:** Tracks system health and recovers from process failures.

2.2 The Role of Shared Memory

All database processes can access a common shared memory region containing:

- **Buffer Pool:** Cache for data pages from disk.
- **Lock Table:** Tracking locking state for concurrency control.
- **Log Buffer:** Staging log records before they are forced to disk.
- **Cached Query Plans:** Pre-compiled plans to speed up repeated queries.

3 Synchronization & Mutual Exclusion

Since multiple processes access shared memory structures, **mutual exclusion** is critical.

Latches vs. Locks

Latches are short-duration, low-overhead mutual exclusion mechanisms implemented via atomic hardware instructions (*test-and-set*, *compare-and-swap*). Unlike database locks, they ensure integrity of internal data structures like the lock table.

3.1 Direct Lock Table Access Workflow

To minimize overhead, server processes often update the lock table directly.

Lock Acquisition

1. Acquire **mutex (latch)** on lock table.
2. Check if lock is available.
3. Update lock table (allocate or queue).
4. Release mutex (latch).

Lock Release

1. Acquire **mutex (latch)** on lock table.
2. Remove entry for the released lock.
3. Process and grant pending requests.
4. Release mutex (latch).

4 Data Servers & Client-Side Caching

Data servers offload heavy computations to client machines, which is particularly useful for CAD or parallel storage systems handling JSON/XML documents.

4.1 Optimization Strategies to Reduce Latency

Network latency (round-trip time) often exceeds local memory access by orders of magnitude.

- **Prefetching:** Sending anticipated data items before they are requested.

- **Data Caching:** Retaining data at the client across multiple transactions. Requires **Cache Coherency** (ensuring data isn't stale).
- **Lock Caching:** Holding locks locally to allow zero-communication access until the server "calls back" the lock for another client.
- **Adaptive Granularity:** Using multi-granularity locking (e.g., page locks) to reduce requests. Includes *Lock De-escalation* to switch to finer locks when contention rises.